



Machine Learning for Petroleum Engineers Using Python

Módulo 7 · Clase 1

Lo Que Antes Necesitaba un Programador



Carlos Enrique Mosquera Trujillo



SLB Ecuador

Dónde Estamos: Cambia la Pregunta

✓ Hasta aquí:

- ✓ Aprendieron a **plantear** un problema de datos
- ✓ A **medir** si un modelo sirve, contra un rival simple
- ✓ A **desconfiar** del dato antes que del modelo
- ✓ Y a entregar una **banda**, no un número solo

Todo eso es criterio de ingeniería. No caduca.

Y como siempre: nada se da por sabido. Nadie tiene que haber programado antes.

▶▶ Desde hoy:

- Ya no preguntamos «¿qué modelo uso?»
- Preguntamos «¿cómo construyo la **herramienta que hace falta?**»
- Y la construimos **dirigiendo a un agente**, no escribiendo cada línea
- ★ Al final de hoy, cada uno tiene una **aplicación con un link**

Ustedes Ya Dirigen Trabajo Técnico

Cuando le encargan un trabajo a un contratista, ya hacen exactamente lo que vamos a hacer hoy con un agente:

1 • Especifican

No dicen «arregle el pozo». Dicen qué, con qué materiales, para cuándo y cómo se sabe que quedó bien.

2 • Reciben y revisan

No firman el acta sin mirar. Revisan contra lo que pidieron, no contra lo que les entregaron.

3 • Responden

Si el trabajo sale mal, la firma es de ustedes. El contratista no va a la reunión.

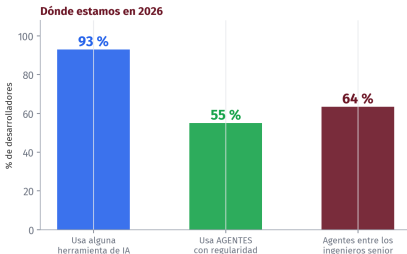
Un agente de código es un contratista muy rápido, muy barato y muy seguro de sí mismo. Todo lo que ya saben sobre dirigir contratistas aplica — incluida la parte de que la firma sigue siendo suya.

Esto Ya Pasó

Cómo se escribe código en 2026, con números

o – 15 min

La Foto de 2026



Y el dato que sorprende

**Los ingenieros
MÁS SENIOR
son los que más
usan agentes**

*No es una herramienta para principiantes.
Es una herramienta que rinde más
cuanto más criterio tiene quien la usa.*

Esto ya no es una tendencia ni una moda: es cómo se trabaja. El **93 %** de los desarrolladores usa alguna herramienta de IA, y más de la mitad usa **agentes** con regularidad.

El Dato que le Conviene a Esta Sala

Los ingenieros más senior son los que más usan agentes: 63,5 % contra 55 % del promedio.

No es una herramienta para el que no sabe. Es una herramienta que **rinde más cuanto más criterio tiene quien la usa** — porque el que sabe reconoce al instante cuándo la respuesta está mal.

Y hay un segundo dato en la misma dirección: quien usa agentes tiene **el doble de probabilidad** de estar entusiasmado con ellos. Quien no los usa, el doble de escéptico. ***El escepticismo, en esto, se cura***

usándolo. No leyendo sobre ello.

Lo que No Dicen los Titulares

Todo el mundo lo usa. Pero casi nadie le cree del todo, y eso también está medido:

LA CONFIANZA

Solo el **3 %** confía plenamente en lo que produce.

El **46 %** desconfía de su precisión, contra un 33 % que confía.

LA FRUSTRACIÓN #1

El **66 %** dice que lo que más le molesta son las «soluciones casi correctas, pero no del todo».

Y el **45 %** dice que depurarlas tarda **más** que escribir el código a mano.

«Casi correcto, pero no» es el problema central de esta clase — y es justo donde hace falta alguien que sepa del tema.

El Cuello de Botella se Movi 



El ancho es el tiempo. Lo rojo es donde se atasca el trabajo.

Escribir c digo dej  de ser lo caro. Lo caro ahora es **verificar**: los cambios hechos con IA esperan m s tiempo en revisi n, y meten m s errores si nadie los mira.

Y Por Eso Ustedes Valen Más, No Menos

Lo que se volvió barato es *escribir*. Lo que sigue siendo escaso es saber si el resultado tiene sentido.

Un agente puede escribir en treinta segundos el código que ajusta una curva de declinación. Lo que no puede hacer es saber que ese campo tuvo un *workover* en 2018, que el medidor estuvo descalibrado seis meses, o que ese pozo no debería estar en la muestra.

Eso lo saben ustedes. Y hasta hoy, para convertir ese conocimiento en una herramienta, había que esperar a que alguien de sistemas tuviera tiempo. ***Eso es lo que se acabó. Y de eso se trata este módulo.***

Acotar el Problema

LO QUE SÍ VAMOS A HACER

- ✓ Gráficas que se entienden solas
- ✓ Dirigir un agente para que las haga
- ✓ **Verificar** lo que devuelve
- ✓ Convertirlo en una app con link

LO QUE NO

- ✗ Convertirlos en programadores
- ✗ Enseñar a memorizar sintaxis
- ✗ Instalar nada: todo corre en Colab
- ✗ Modelos nuevos — hoy es sobre **construir**, no sobre predecir

Regla de la casa para hoy: si algo no lo entienden, es culpa del material, no de ustedes. Pregunten en el momento.

Primera Victoria

Que sus datos se vean como lo que valen

15 – 45 min

Empecemos por Algo que Usan Todas las Semanas

Todos hacen reportes. Y todos han visto la misma escena: un gráfico correcto, con los números bien, que nadie entiende en la reunión — y hay que explicarlo hablando.

Un gráfico que hay que explicar es un gráfico que falló.

Una gráfica no es un adorno del análisis: es el argumento.

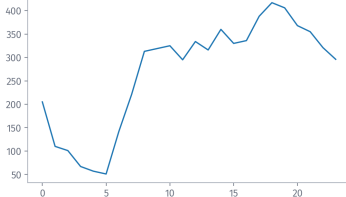
Es lo único que la mayoría de la gente va a ver de todo su trabajo. El modelo, los datos y las tres semanas de esfuerzo llegan a la gerencia convertidos en una imagen.

Y es el mejor lugar para empezar a dirigir un agente: se ve el resultado de inmediato, y si está mal, se nota.

Los Mismos Datos, Dos Veces

Los mismos datos, los mismos números. Solo cambia si se entiende.

Lo que sale por defecto



La misma información, comunicando



Mismos datos, mismos números, misma librería. A la izquierda hay que explicar; a la derecha, la respuesta salta sola.

Qué Cambió, Exactamente

No fue magia ni buen gusto. Fueron **cinco decisiones**, y todas se le pueden pedir a un agente:

1. **Barras en vez de línea.** Una línea sugiere continuidad; las horas son categorías separadas.
2. **Un solo color con sentido.** Gris todo, rojo lo que importa. El color señala, no decora.
3. **Los ejes dicen qué son** — «hora del día», «viajes» — en el idioma del que lee.
4. **El hallazgo, escrito sobre la figura.** Si la conclusión es «el pico es a las 18h», eso va en la lámina, no en la boca del presentador.
5. **Se quitó todo lo demás.** Sin marco, sin cuadrícula de más, sin decimales que nadie necesita.

Ninguna de las cinco es programación. Las cinco son criterio de comunicación — y por eso las tienen que decidir ustedes, no el agente.

Qué Es Seaborn y Por Qué Existe

Matplotlib es la librería con la que Python dibuja. Puede hacer todo, pero hay que decirle todo: cada color, cada margen, cada etiqueta. Por eso lo que sale «por defecto» se ve así de crudo.

Seaborn se construyó encima. Trae decisiones de diseño ya tomadas por gente que estudia cómo se leen los gráficos, y entiende de tablas: uno le dice qué columna va en cada eje y él se encarga del resto.

Y ya viene instalado en Colab. No hay nada que instalar.

Además trae datasets de práctica listos, y de ahí salen los datos de hoy: una línea y ya tienen 6.433 viajes de taxi con los que trabajar.

Las Tres Líneas con las que Arranca Todo

```
import seaborn as sns
```

```
taxis = sns.load_dataset("taxis")  
taxis.head()
```

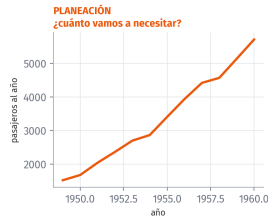
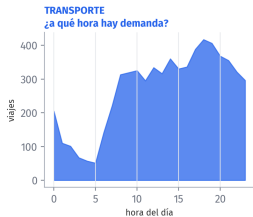
Eso es todo. `load_dataset` baja los datos y los deja en una tabla — lo que en Python se llama un **DataFrame**: filas y columnas, como una hoja de cálculo, pero manejable con código.

Los datos de hoy: 6.433 viajes de taxi de Nueva York, con hora de salida y llegada, distancia, tarifa, propina, forma de pago y zonas.

¿Por qué taxis y no pozos? Porque hoy la habilidad es lo que importa, y quiero que se vea que sirve igual para cualquier problema. En el proyecto final cada uno trae el suyo.

La Misma Habilidad, Tres Problemas Distintos

La misma habilidad, tres problemas que no se parecen en nada



Transporte, gestión presupuestal y planeación de demanda. Tres preguntas que no se parecen en nada — y la técnica para responderlas es la misma.

Su Primer Encargo al Agente

🕒 10 min

Abren el cuaderno de la clase y peguen esto en el panel de Gemini. Fíjense en cómo está escrito: no dice «hazme un gráfico bonito».

```
Tengo un DataFrame 'taxis' de seaborn con viajes de taxi de Nueva York.  
Columnas: pickup y dropoff (fecha y hora), distance, fare, tip, total, payment,  
pickup2one, pickup6orough.  
OBJETIVO: quiero saber a qué hora del día hay más demanda de taxis.  
RESTRICCIONES: - usa seaborn - el gráfico va en un reporte para gerencia: tiene que  
entenderse sin que yo lo explique - ejes con nombre y unidades, en español  
CRITERIO DE ACEPTACIÓN: - la hora pico marcada y legible de un vistazo - avísame si  
hay horas sin ningún viaje registrado
```

Compárenlo con pedir «grafica los viajes». Volvemos sobre por qué en el bloque que sigue.

Cómo se le Pide Bien

Qué es un agente, y los cuatro fundamentos

45 – 70 min

Tres Cosas Distintas que la Gente Llama «la IA»

- **Autocompletado.** Sugiere el final de la línea que están escribiendo. Útil, pero no entiende el problema: entiende el patrón.
- **Asistente.** Uno le pregunta, él contesta con un texto o un bloque de código. Hay que copiarlo, pegarlo, correrlo y ver qué pasó. *Él no ve el resultado.*
- **Agente.** Escribe el código, **lo ejecuta, mira lo que salió** — incluidos los errores — y **lo corrige solo.** Repite hasta que funciona.

La diferencia no es que el agente adivine mejor. Es que itera.

Y por eso, cuando se equivoca, se equivoca con mucha más convicción: ya probó, ya corrigió, y les entrega algo que *corre*.

Los Cuatro Fundamentos

Las cuatro cosas que se le dicen a cualquiera a quien uno le encarga un trabajo:

1 • OBJETIVO

Qué pregunta quiero responder. **No** qué gráfico quiero: qué quiero *saber*.

3 • RESTRICCIONES

Qué herramientas usar, en qué idioma, qué **NO** hacer. Sin esto, inventa lo que le parezca.

2 • CONTEXTO

Qué datos hay, cómo se llaman las columnas, en qué unidades, y para quién es el resultado.

4 • CRITERIO DE ACEPTACIÓN

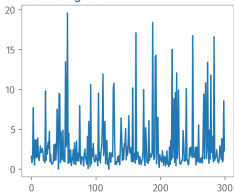
Cómo sabemos que quedó bien. Es el que casi nadie escribe, y el que más cambia el resultado.

El cuarto es el que convierte un pedido en un encargo de ingeniería.

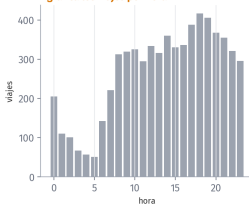
La Escalera, en Vivo

La calidad de la salida es la calidad de la especificación

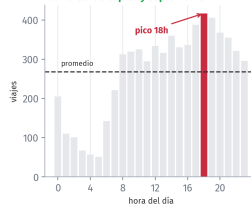
«hazme un gráfico»



«grafica los viajes por hora»



«...marcando el pico y el promedio»



El mismo agente, los mismos datos, tres pedidos distintos. **La calidad de la salida es la calidad de la especificación.**

La Plantilla que se Llevan

Guárdenla. Sirve para pedir una gráfica, un análisis o una aplicación entera:

CONTEXTO: tengo [qué datos], con columnas [cuáles] en [qué unidades]. Es para [quién / qué reunión].

OBJETIVO: quiero responder [qué pregunta].

RESTRICCIONES: - usa [qué librerías] - [qué NO hacer] - en español, ejes con unidades

CRITERIO DE ACEPTACIÓN: - [qué tiene que verse / cumplirse] - avísame si [qué problema podría tener el dato]

Ese último renglón — «avísame si...» — es el que convierte al agente en un aliado para encontrar problemas, en vez de en alguien que los tapa.

Segundo Encargo: Una Pregunta de Operación

🕒 10 min

Ahora una pregunta que un jefe de operaciones hace de verdad: *¿cuándo conviene tener más gente en la calle?*

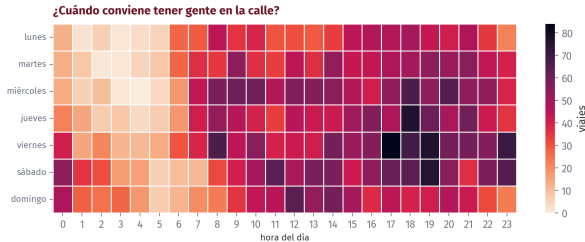
CONTEXTO: el mismo DataFrame 'taxis'. La columna pickup tiene fecha y hora de inicio del viaje.

OBJETIVO: saber en qué combinaciones de día de la semana y hora se concentra la demanda, para decidir turnos.

RESTRICCIONES: - un mapa de calor con seaborn (días en filas, horas en columnas)
- días en español y en orden lunes a domingo - no uses colores rojo-verde (hay daltónicos en la sala)

CRITERIO DE ACEPTACIÓN: - que se pueda señalar con el dedo el peor momento - dime cuál es ese momento y cuántos viajes tiene

Lo que Debería Salir



El momento de más demanda es el **viernes a las 17h**. Con eso ya se puede decidir un turno — y esa decisión no la tomó el agente: la tomaron ustedes al preguntar bien.

Dónde Falla

Y por qué sigue haciendo falta un ingeniero

70 – 95 min

Ahora la Parte Incómoda

Hasta acá todo salió bien, y esa es justamente la trampa. Un agente que funciona bien noventa veces genera confianza — y a la vez noventa y uno hace algo que parece igual de bueno y está mal.

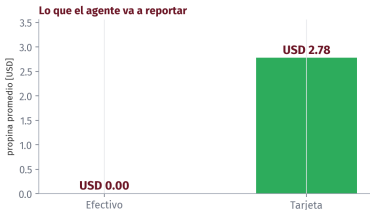
Vamos a hacer una pregunta perfectamente razonable sobre los mismos datos:

«¿Los pasajeros que pagan en efectivo dejan menos propina que los que pagan con tarjeta?»

Es el tipo de pregunta que se hace para decidir una política de medios de pago. Y tiene consecuencias reales en dinero.

El agente la responde en veinte segundos, con un gráfico impecable.

Un Análisis Impecable



«Quien paga en efectivo nunca deja propina»

Y es falso

De los 1812 viajes en efectivo, exactamente 0 tienen propina.

No es que no la dejen: el taxímetro solo registra la propina cuando va en la tarjeta. La de efectivo va al bolsillo del conductor y nunca entra al sistema.

Un cero que no es un cero: es una **AUSENCIA**

El cálculo está bien hecho. El gráfico está bien hecho. La conclusión es **falsa**.

Un Cero que No Es un Cero

De los **1.812** viajes pagados en efectivo, los que tienen propina registrada son **exactamente cero**. No 3, no 50: cero.

Ese número perfecto es la pista. En datos reales, nada es exactamente cero por casualidad.

El taxímetro solo registra la propina cuando va en la tarjeta.

La propina en efectivo va directo al bolsillo del conductor y nunca entra al sistema. La columna no dice «no dejó propina»: dice **«no tengo ese dato»**.

Y si alguien decide con eso — por ejemplo, empujar el pago con tarjeta para «recuperar» propinas — toma una decisión de negocio sobre una ausencia de datos.

Esto Ya lo Vivieron

Es exactamente el mismo problema que encontramos en el módulo del agua: campos que aparecían con **cero agua producida** durante veinticinco años.

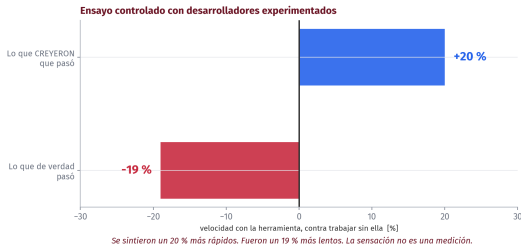
No es que no produjeran agua. Es que el regulador **no la empezó a exigir hasta el año 2000**. Un cero que era una ausencia.

Distinto dominio, distinto dato, distinto país — el mismo error.

Y en los dos casos, lo que lo detectó no fue una técnica: fue alguien que conocía el negocio y dijo «esto no puede ser».

El agente no tiene esa alarma. Ustedes sí.

Y Cuidado con la Sensación



Un ensayo controlado con desarrolladores experimentados: se sintieron un **20 % más rápidos** y fueron un **19 % más lentos**. No es un argumento contra la herramienta — es uno a favor de medir.

La Lista de Verificación

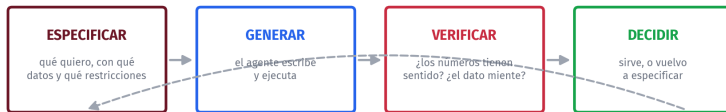
Cuatro preguntas para cualquier resultado que devuelva un agente. Toman dos minutos y evitan la mayoría de los desastres:

- ❓ **¿Cuántas filas entraron y cuántas salieron?** Si el agente descartó la mitad de los datos sin avisar, todo lo demás sobra.
- ❓ **¿Hay ceros, vacíos o números redondos sospechosos?** Un cero perfecto casi siempre es una ausencia disfrazada.
- ❓ **¿El resultado tiene sentido físico o de negocio?** Si dice que un pozo produce más que el campo entero, no hace falta revisar el código.
- ❓ **¿Qué decisión se tomaría con esto, y qué pasa si está mal?** Si la respuesta es «nada grave», sigan. Si no, verifiquen a mano.

Ninguna de las cuatro exige saber programar. Las cuatro exigen saber del negocio.

El Ciclo Real de Trabajo

El trabajo de ustedes está en las cajas de los extremos



y si no sirve, se vuelve a especificar — eso es el trabajo

El agente hace la caja del medio. Las dos de los extremos — especificar y decidir — son de ustedes, y no se delegan.

Tercer Encargo: Pónganle Trampas

🕒 8 min

Ahora usen al agente para lo contrario: para **buscar** problemas en vez de taparlos.

CONTEXTO: DataFrame 'taxis' de seaborn.

OBJETIVO: quiero saber si estos datos tienen algún problema ANTES de sacar conclusiones.

Hazme una revisión de calidad que incluya: - filas totales y filas con datos faltantes - para cada columna numérica: mínimo, máximo, y cuántos valores son exactamente cero - si alguna combinación de categorías tiene SIEMPRE el mismo valor (eso suele ser un artefacto del sistema que registra, no un hecho real)

CRITERIO DE ACEPTACIÓN: - una tabla que yo pueda leer en 30 segundos - y una lista de las cosas que te parecen raras, aunque no estés seguro

Este prompt vale por toda la clase: guárdenlo y córranlo con cada dataset nuevo, antes de cualquier análisis.

De la Gráfica a la App

Que otro lo pueda usar sin llamarlos

95 – 120 min

El Salto que Falta



El salto que separa un análisis de una herramienta

Mientras el análisis viva en una celda, cada vez que alguien quiera ver otra cosa tiene que llamarlos. Una app se la mandan y se acabó.

Qué Es Gradio

Una librería que convierte una función de Python en una página web con controles. Uno le dice: «esta función recibe una hora y devuelve un gráfico», y Gradio arma la interfaz sola.

No hay que saber nada de páginas web. Y en Colab hace algo muy útil: genera un **link público** que se le puede mandar a cualquiera.

LO QUE HAY QUE SABER DEL LINK

Funciona **mientras el cuaderno esté corriendo**. Si cierran Colab o se desconecta, el link deja de responder. Sirve perfecto para mostrar algo en una reunión; no sirve como sistema permanente.

Cómo se hace permanente lo vemos en la Clase 3. Hoy nos alcanza y sobra.

La App Más Chica que Sirve

```
import gradio as gr

def ver_demanda(dia):
    d = taxis[taxis.dia_nombre == dia]
    return grafico_de_horas(d)

demo = gr.Interface(
    fn=ver_demanda,
    inputs=gr.Dropdown(DIAS, label="Día"),
    outputs=gr.Plot(label="Demanda por hora"),
    title="Demanda de taxis por día y hora")

demo.launch(share=True)
```

Tres piezas: la **función** que hace el trabajo, los **controles** de entrada y salida, y `launch(share=True)` que publica el link.

Su Turno: Construyan la Herramienta

🕒 20 min

CONTEXTO: en este cuaderno ya tengo el DataFrame 'taxis' cargado y una función que grafica la demanda por hora.

OBJETIVO: convertirlo en una app de Gradio que pueda mandarle por link al jefe de operaciones.

La app tiene que dejarle elegir: - el día de la semana - el barrio de origen (pickup_{orough})

y mostrarle el gráfico de demanda por hora para esa combinación, más una línea de texto que diga cuál es la hora pico y cuántos viajes tiene.

RESTRICCIONES: - usa gradio, con launch(share=True) - todo en español - si la combinación elegida no tiene datos, que lo diga con un mensaje claro en vez de fallar

CRITERIO DE ACEPTACIÓN: - me tiene que dar un link que yo pueda abrir - y no se puede caer si elijo un barrio sin viajes

El Entregable del Módulo

Empiecen a pensarlo desde hoy. En la última clase cada uno presenta:

Una aplicación que resuelva un problema real de su trabajo, construida dirigiendo a un agente, con su link funcionando.

Y con tres cosas más, que valen tanto como la app:

- **Los prompts que usaron.** Queremos ver cómo dirigieron, no solo qué salió.
- **Un error que el agente cometió,** y cómo lo detectaron.
- **Qué decisión se toma** con la herramienta, y quién la toma.

Traigan su propio dato. Si no pueden por confidencialidad, lo hablamos — hay maneras, y son parte de la Clase 3.

Cierre

Lo Que Aprendimos Hoy

💡 Conceptos:

- ✓ Autocompletado, asistente y **agente** no son lo mismo
- ✓ Una gráfica es un **argumento**, no un adorno
- ✓ Los **cuatro fundamentos**: objetivo, contexto, restricciones, criterio de aceptación
- ✓ Un **cero perfecto** casi siempre es una ausencia
- ✓ La **lista de verificación** de cuatro preguntas

🔧 Herramientas:

- seaborn y load_dataset
- El panel de Gemini de Colab
- gradio y launch(share=True)
- La **plantilla de prompt** — la que más van a usar

EL NÚMERO DEL DÍA

1.812 y 0. Los viajes en efectivo, y los que tenían propina registrada. El agente no vio nada raro.

El Resultado Negativo de Hoy

La regla de este curso es que al menos un resultado negativo se dice en voz alta. Hoy hubo dos, y los dos están medidos:

- ✗ **El agente entregó un análisis correcto con una conclusión falsa**, y no dio ninguna señal de alarma. El código estaba bien; el dato mentía.
- ✗ **En un ensayo controlado, desarrolladores experimentados fueron 19 % más lentos** usando estas herramientas, mientras se sentían 20 % más rápidos.

Ninguno de los dos es razón para no usarlas. Los dos son razón para medir en vez de confiar en la sensación — que es exactamente lo que venimos haciendo todo el diplomado.

Si Se Llevan Una Sola Cosa

El agente escribe el código. Ustedes responden por el resultado.

Y eso no es una carga: es la razón por la que su criterio vale más ahora que hace tres años. Escribir se volvió barato; saber si el resultado tiene sentido, no.

La habilidad que se llevan hoy no es «usar Gemini». Es **especificar bien y verificar siempre** — y eso sirve con cualquier herramienta que salga el año que viene.

Nadie en esta sala escribió una línea de código hoy. Y todos se van con una aplicación funcionando.

Lo Que Sigue

- ➔ **Clase 2 — Que responda, y que no mienta.** Cómo se sirve un modelo de verdad: qué entra, qué sale, qué pasa cuando le mandan un dato raro. Y el error más caro de todos: el que funciona en el cuaderno y falla en producción.
- ➔ **Clase 3 — Los peligros, con nombre y apellido.** Confidencialidad — qué se puede pegar en un chat de IA y qué no —, librerías que el agente inventa, y código que corre y está mal.
- ➔ **Clase 4 — Su proyecto.** Cada uno con su problema y su herramienta, presentada al grupo.

✓ TAREA PARA LA PRÓXIMA

Piensen en **un** problema de su trabajo donde hoy alguien pierde tiempo con una hoja de cálculo. Ese es su proyecto. Tráiganlo escrito en dos frases.

Datos y Referencias

-  **Adopción y confianza:** Stack Overflow Developer Survey y su análisis de la brecha de confianza (2026); *State of Code*, Sonar (enero de 2026).
-  **Productividad medida:** ensayo controlado de **METR** con desarrolladores experimentados; informes de adopción de agentes 2026.
-  **Datos:** taxis, healthexp y flights, de los conjuntos de práctica que trae **seaborn**. Se cargan con una línea y no hay que bajar nada.
-  Las figuras y **todas** las cifras de estas láminas salen de `figuras.py`. Nada está escrito a mano.

Como siempre: corren el script y tiene que dar exactamente lo mismo. Si no da, es un error nuestro y queremos saberlo.

¡Gracias!

Carlos Enrique Mosquera Trujillo

✉ cmosquerat@unal.edu.co

Machine Learning for Petroleum Engineers Using Python

Módulo 7 · Clase 1 · SLB Ecuador · UDLA · 2026